CMP1020 – Học sâu

BÀI 3: MẠNG NEURON (NEURAL NETWORK)

Giảng viên: TS. Bùi Danh Hường

MỤC TIÊU BÀI HỌC

- Hiểu cấu tạo và nguyên lý hoạt động của mạng neuron nhân tạo.
- Nắm vững các hàm kích hoạt (Activation functions) phổ biến.
- Hiểu và tính toán được thuật toán Lan truyền ngược (Backpropagation).
- Thiết kế được mạng neuron cho bài toán Phân lớp và Hồi quy.

CẤU TRÚC BÀI HỌC

Lý thuyết

Khái niệm cơ bản & Cấu trúc mạng.

Thuật toán

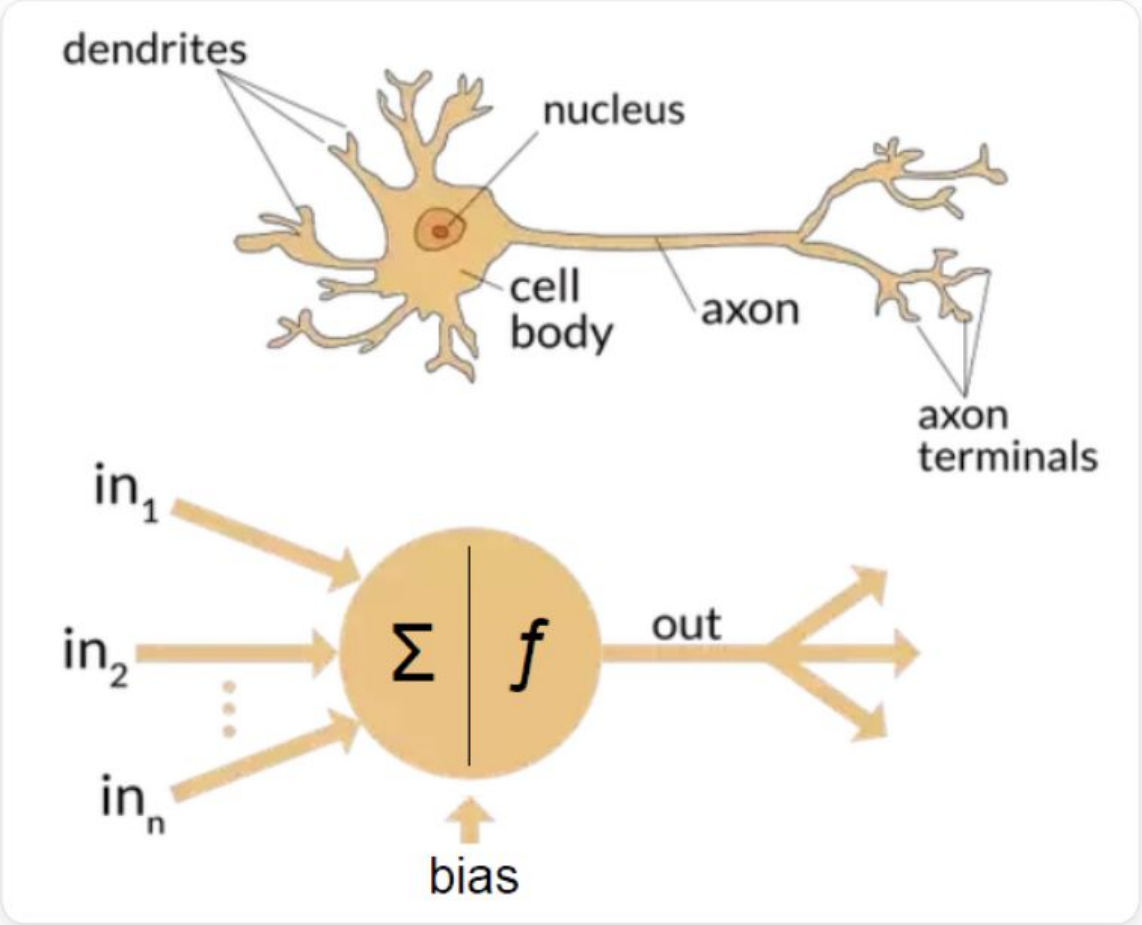
Lan truyền ngược (Backpropagation).

Ứng dụng

Phân lớp ảnh & Dự đoán giá trị.

CẢM HỨNG TỪ SINH HỌC

- **Não bộ:** Gồm hàng tỷ neuron kết nối với nhau qua các khớp thần kinh (synapse).
- **Cơ chế hoạt động:**
  Nhận tín hiệu điện → Xử lý thông tin → Truyền đi (nếu tín hiệu đủ mạnh).
- **Mô phỏng:** Mạng neuron nhân tạo (ANN) bắt chước cơ chế này để giúp máy tính có khả năng "học" tri thức từ dữ liệu.



Sự tương đồng giữa Neuron sinh học và Neuron nhân tạo

MÔ HÌNH TOÁN HỌC CỦA MỘT NEURON

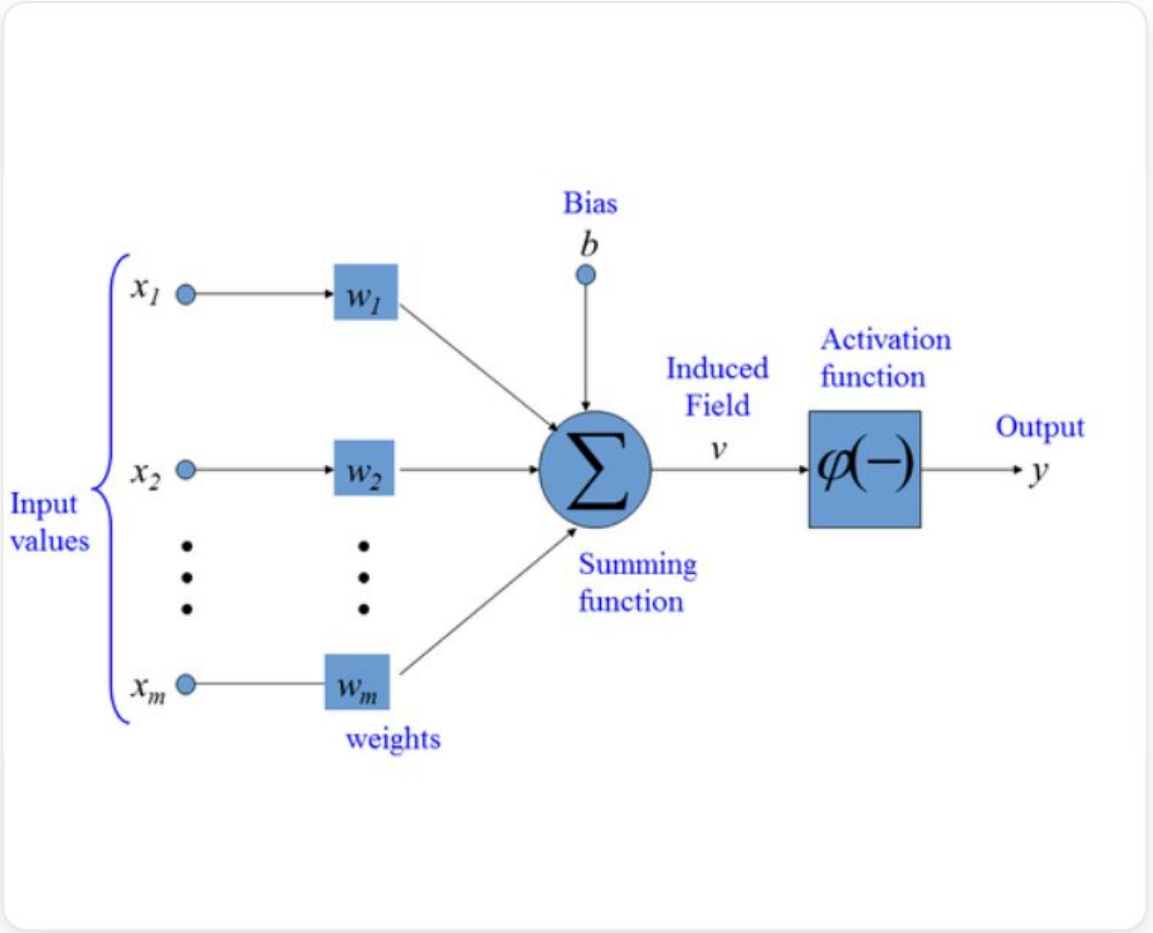
Một neuron nhân tạo thực hiện **2 bước tính toán** chính:

1. Tổng hợp thông tin (Sum): Tính tổng có trọng số của các đầu vào cộng với bias.

$$z = \sum w_i \cdot x_i + b$$

2. Kích hoạt (Activation): Đưa kết quả z qua một hàm phi tuyến f.

$$\text{Output} = fz$$



Sơ đồ luồng tín hiệu trong một Neuron nhân tạo

THÀNH PHẦN 1: TRỌNG SỐ (WEIGHTS – w)

Định nghĩa

Hệ số thể hiện **độ quan trọng** của kết nối giữa các neuron.

Nó quyết định tín hiệu đầu vào sẽ được khuếch đại hay giảm thiểu khi truyền đi.

Ý nghĩa

- **Trọng số lớn:**

  Đặc trưng đầu vào ảnh hưởng **mạnh** đến quyết định.

- **Trọng số âm:**

  Đặc trưng đầu vào **kìm hãm** (ức chế) đầu ra.

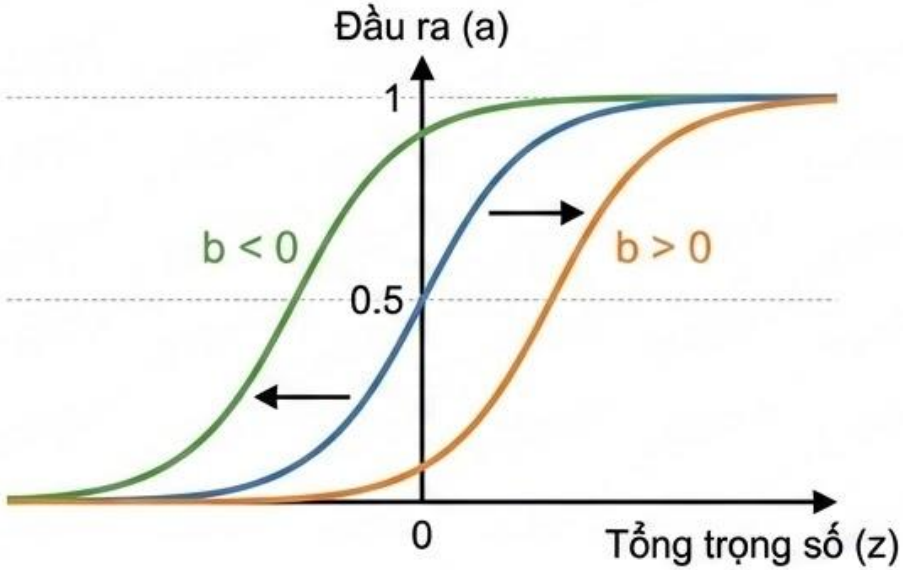
Ví dụ: Mua nhà

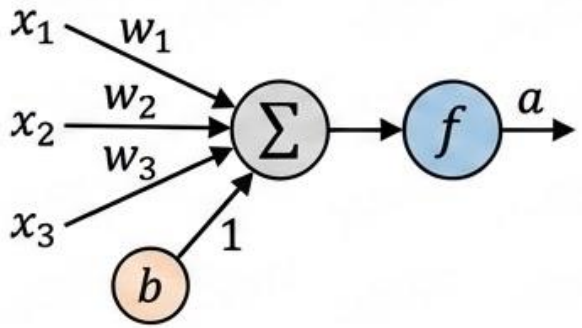
Quyết định: Có mua không?

- **Giá tiền (Cao):**
  Trọng số Âm lớn
  (Giá càng cao → Càng ít muốn mua).
- **Vị trí (Gần):**
  Trọng số Dương
  (Gần trung tâm → Càng muốn mua).

THÀNH PHẦN 2: BIAS (b)

- Định nghĩa: Hệ số chặn, được cộng thêm vào tổng trọng số.
- Ý nghĩa: Giúp dịch chuyển hàm kích hoạt sang trái hoặc phải trên đồ thị.
- Vai trò: Cho phép neuron kích hoạt ngay cả khi tất cả đầu vào bằng 0 (tương tự hệ số w_0 trong hồi quy tuyến tính).





THÀNH PHẦN 3: HÀM KÍCH HOẠT (ACTIVATION)

Vấn đề

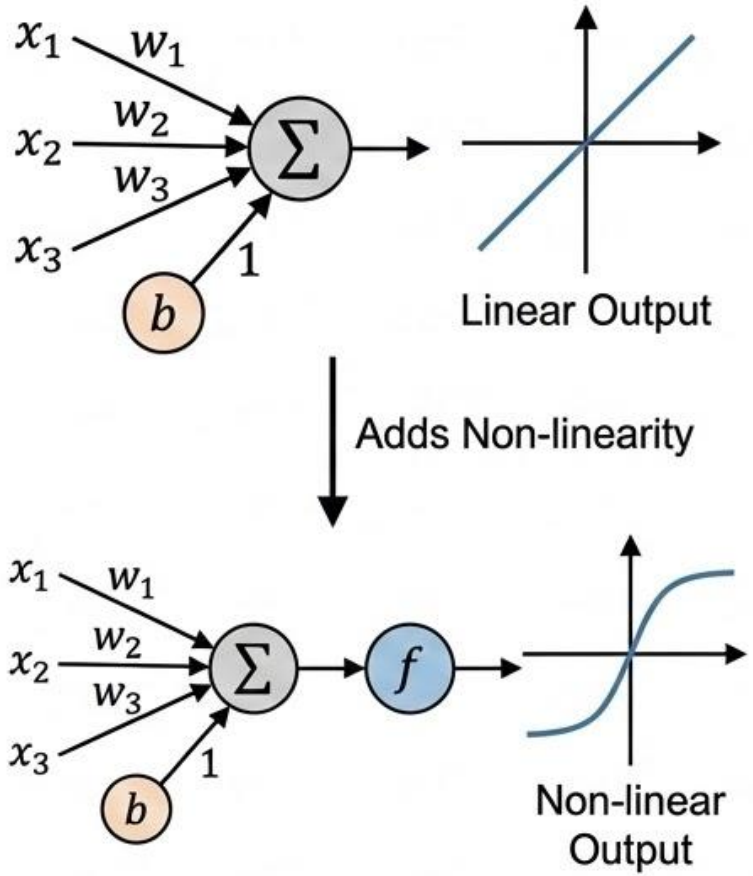
- Nếu chỉ có cộng và nhân ($wx+b$), mạng neuron dù sâu đến đâu cũng chỉ là một hàm tuyến tính.

Giải pháp

- Hàm kích hoạt đưa tính phi tuyến (non-linearity) vào mô hình.

Tác dụng

- Giúp mạng học được các biên quyết định phức tạp (đường cong, đường gấp khúc).



HÀM SIGMOID

Công thức

$$f(x) = \frac{1}{1 + e^{-x}}$$

Đặc điểm

- Đầu ra nằm trong khoảng (0, 1).

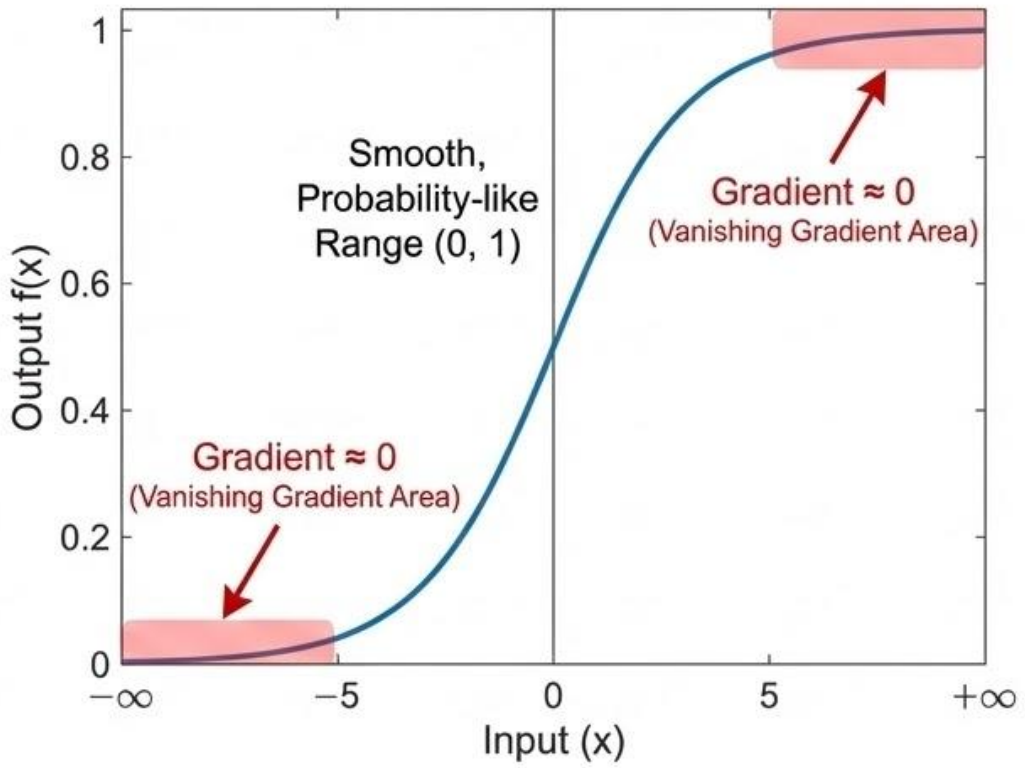
Ưu điểm

- Giống xác suất, mượt mà.

Nhược điểm

- Dễ gặp lỗi 'Vanishing Gradient' (Biến mất đạo hàm) khi đầu vào quá lớn hoặc quá nhỏ.

Minh họa & Vấn đề Vanishing Gradient



HÀM TANH (HYPERBOLIC TANGENT)

Công thức

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Đặc điểm

- Đầu ra nằm trong khoảng (-1, 1).

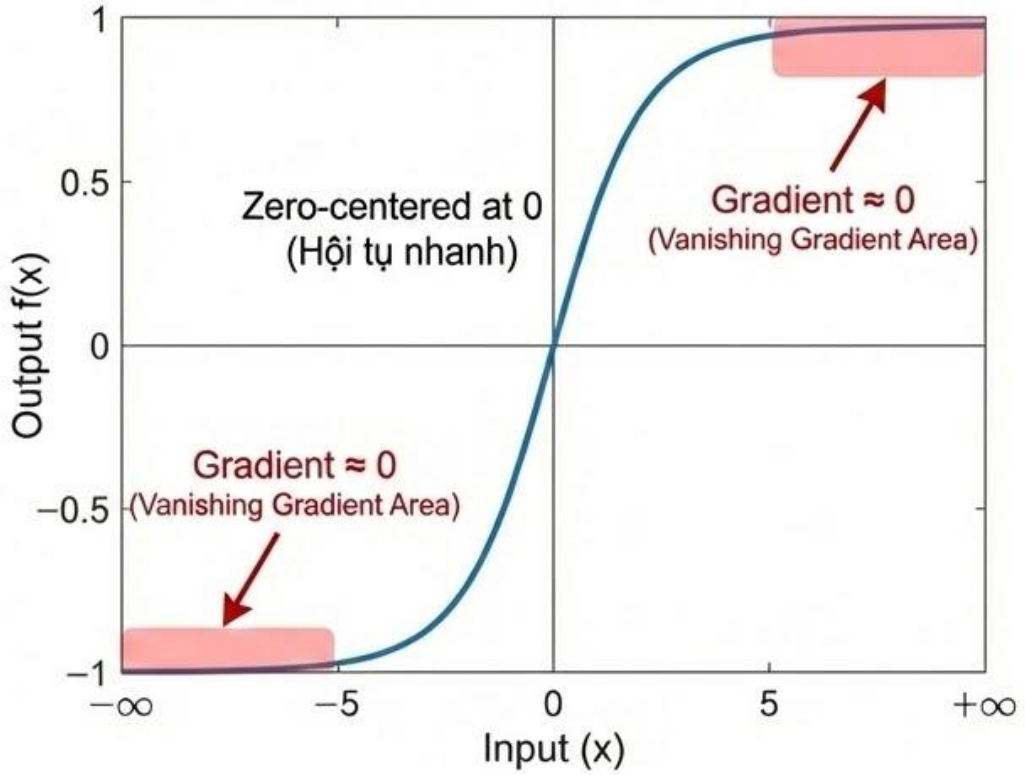
Ưu điểm

- Trung tâm tại 0 (zero-centered), thường hội tụ nhanh hơn Sigmoid.

Nhược điểm

- Vẫn bị Vanishing Gradient.

Minh họa & Vấn đề Vanishing Gradient (Tanh)



HÀM RELU (RECTIFIED LINEAR UNIT)

Công thức

$$f(x) = \max(0, x)$$

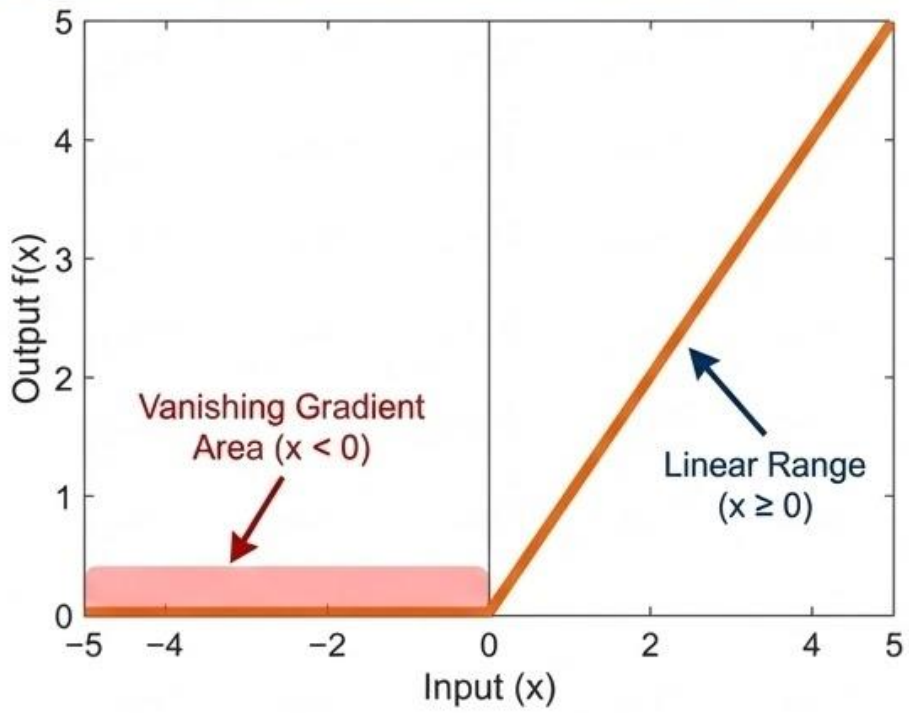
Điều kiện

- Nếu $x < 0$: Output = 0
- Nếu $x \geq 0$: Output = x

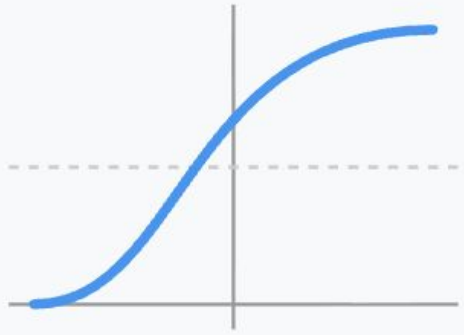
Ưu điểm & Phổ biến

- Tính toán cực nhanh, giảm thiểu Vanishing Gradient.
- Là lựa chọn mặc định cho các lớp ẩn hiện nay.

Minh họa Hàm ReLU

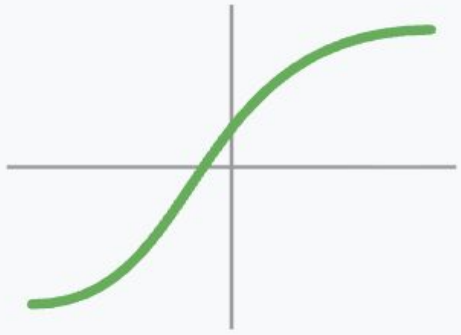


SO SÁNH ĐỒ THỊ CÁC HÀM



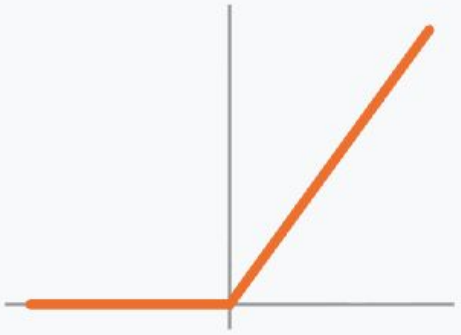
Sigmoid

Range: (0, 1)



Tanh

Range: (−1, 1)



ReLU

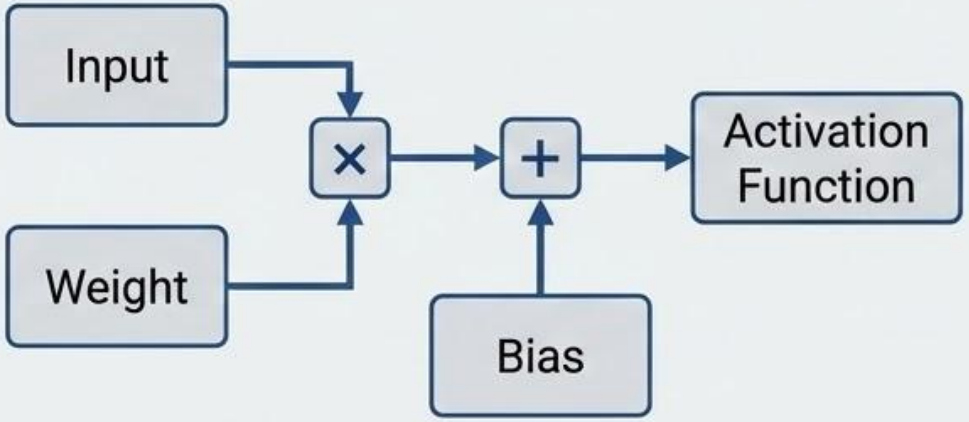
Range: [0, ∞)

Lưu ý: Sự lựa chọn hàm kích hoạt ảnh hưởng rất lớn đến tốc độ tính toán và khả năng hội tụ của mạng (ví dụ: ReLU hội tụ nhanh hơn nhiều so với Sigmoid/Tanh).

TỔNG KẾT PHẦN 1

- **Cơ chế hoạt động của Neuron**

$$\text{Neuron} = (\text{Input} \times \text{Weight} + \text{Bias}) \rightarrow \text{Activation Function}$$



- **Tính Phi Tuyến**



→ Tính phi tuyến là chìa khóa sức mạnh của mạng neuron.

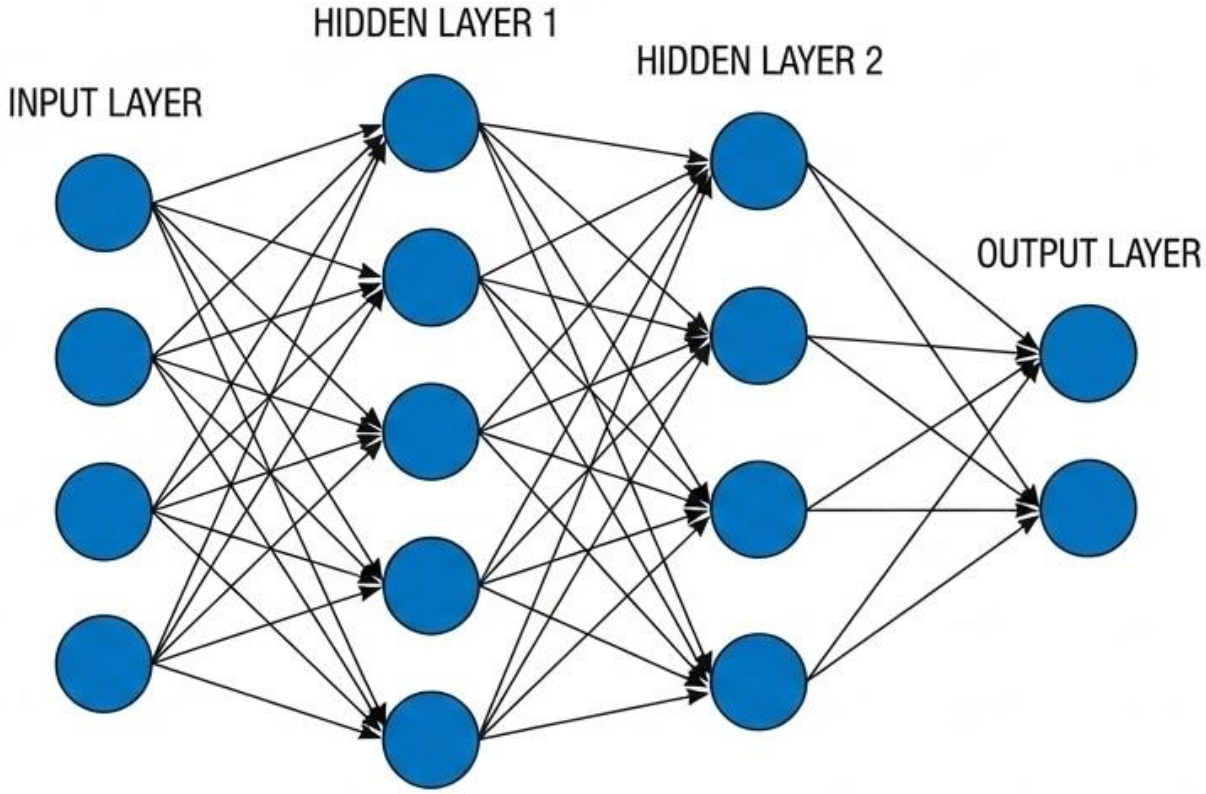
- **Hàm ReLU**



→ **ReLU** là hàm kích hoạt thông dụng nhất cho các lớp ẩn.

KIẾN TRÚC MẠNG (MULTI-LAYER PERCEPTRON)

- Gồm 3 loại lớp xếp chồng lên nhau:
  - **Input Layer:** Lớp đầu vào.
  - **Hidden Layer(s):** Các lớp ẩn.
  - **Output Layer:** Lớp đầu ra.
- **Fully Connected:** Mỗi neuron lớp trước nối với mọi neuron lớp sau.



INPUT LAYER (LỚP ĐẦU VÀO)

Chức năng



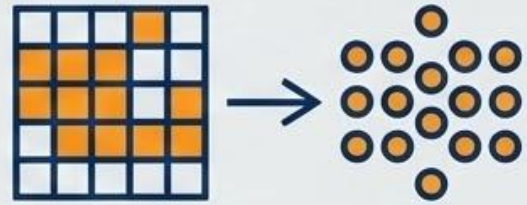
Nơi tiếp nhận dữ liệu thô. Lớp này không thực hiện tính toán, chỉ truyền giá trị.

Số lượng



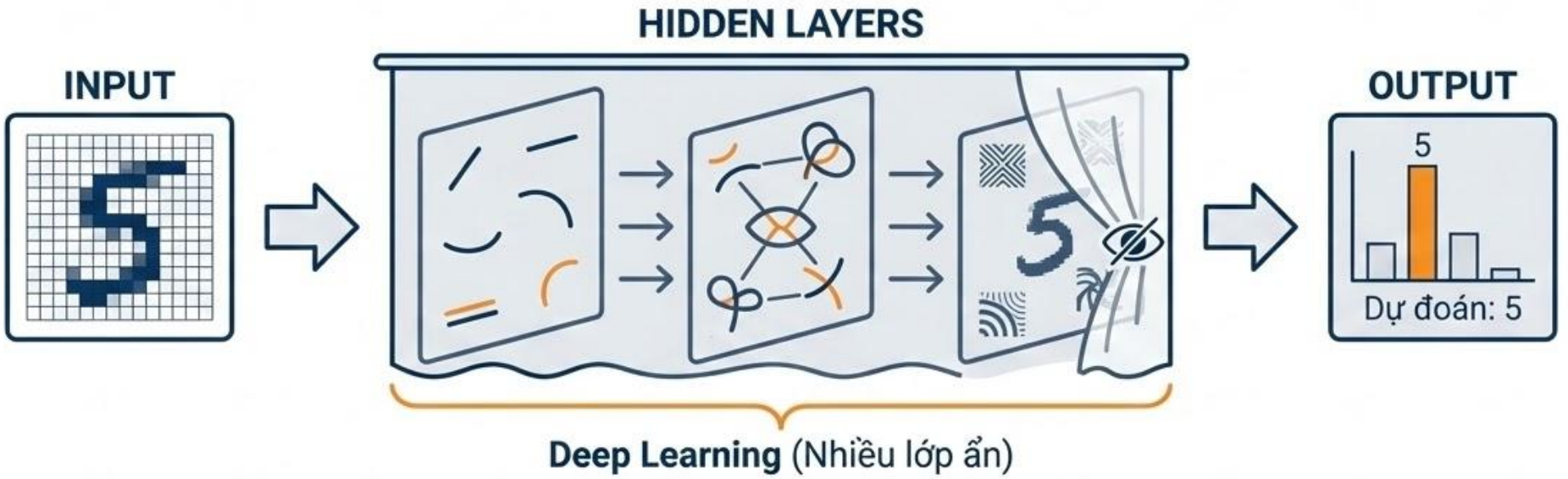
Số neuron = Số chiều (đặc trưng) của dữ liệu.

Ví dụ



Ảnh 28x28 pixels → Cần 784 neuron đầu vào.

HIDDEN LAYER (LỚP ẨN)



Vị trí
Nằm giữa Input và Output.

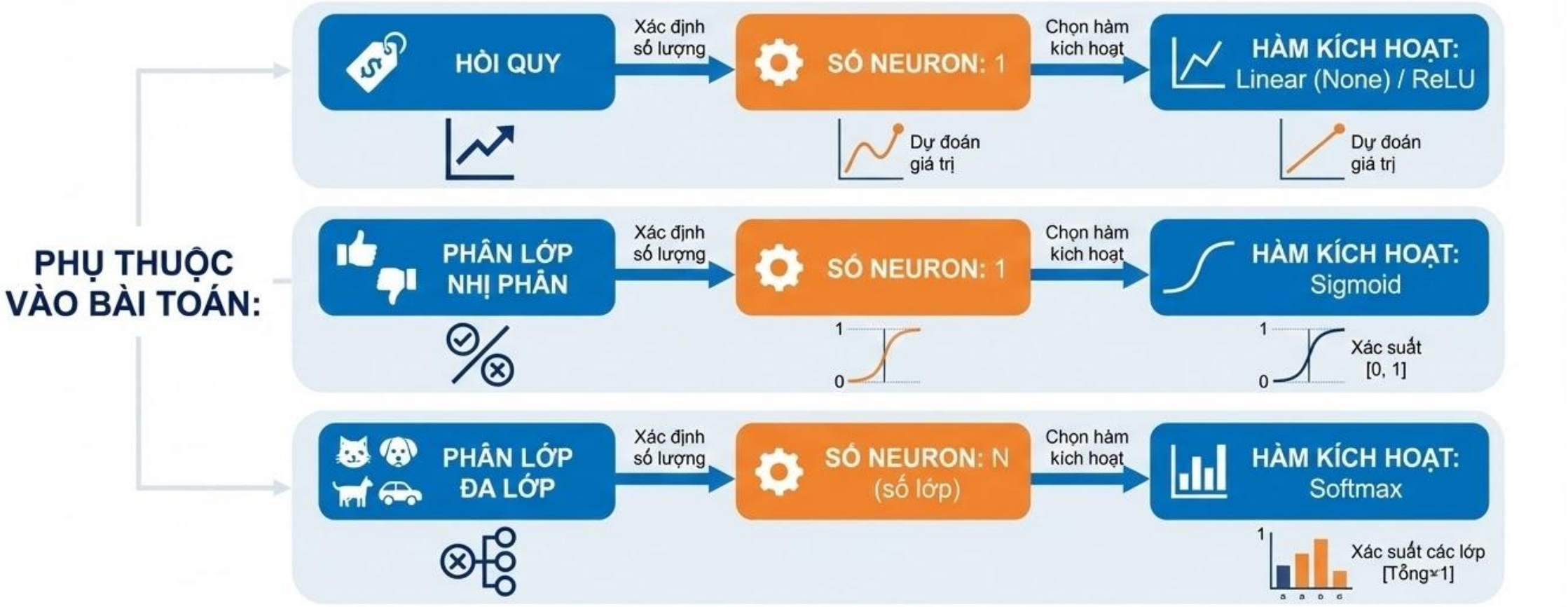
Đặc điểm
"Ẩn" vì người dùng không thấy trực tiếp đầu vào/đầu ra của nó.

Chức năng
Trích xuất đặc trưng. Lớp càng sâu, đặc trưng càng trừu tượng.

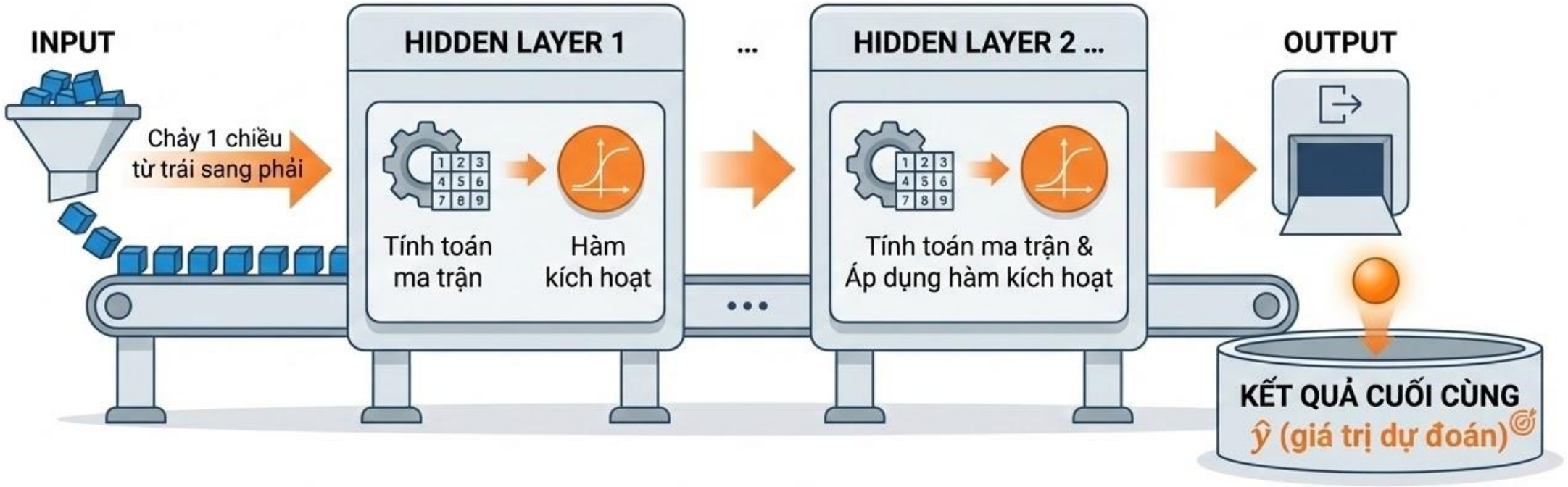
Deep Learning
Deep Learning: Là mạng có nhiều lớp ẩn (Deep).

OUTPUT LAYER (LỚP ĐẦU RA)

- Đưa ra kết quả dự đoán cuối cùng.



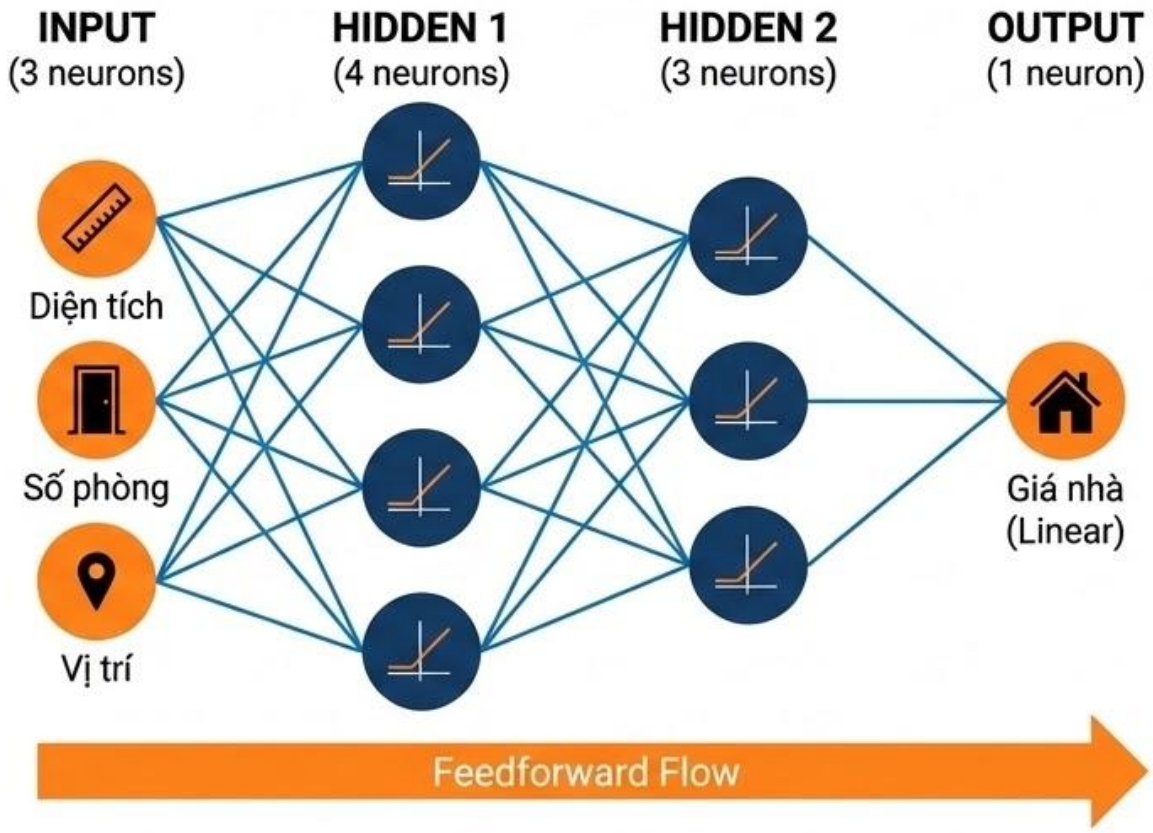
LAN TRUYỀN TIẾN (FEEDFORWARD)



- Dữ liệu chảy 1 chiều từ trái sang phải: $Input \rightarrow Hidden_1 \rightarrow Hidden_2 \rightarrow \ldots \rightarrow Output$
- Tại mỗi lớp, thực hiện **tính toán ma trận** và áp dụng **hàm kích hoạt**.
- Kết quả cuối cùng là $\hat{y}$ (giá trị dự đoán).

VÍ DỤ MINH HỌA KIẾN TRÚC

- **BÀI TOÁN: DỰ ĐOÁN GIÁ NHÀ**
- Mạng học cách kết hợp các yếu tố phức tạp để ra giá.
  - **INPUT LAYER:** Diện tích, Số phòng, Vị trí (3 neuron).
  - **HIDDEN LAYER 1:** 4 neuron (ReLU).
  - **HIDDEN LAYER 2:** 3 neuron (ReLU).
  - **OUTPUT LAYER:** Giá nhà (1 neuron, Linear).



VẤN ĐỀ HUẤN LUYỆN MẠNG



Mục tiêu

Tìm bộ trọng số (W) và bias (b) tối ưu.



Đánh giá

Làm sao biết bộ trọng số tốt? → Dùng **Hàm mất mát (Loss Function)**.



Cải thiện

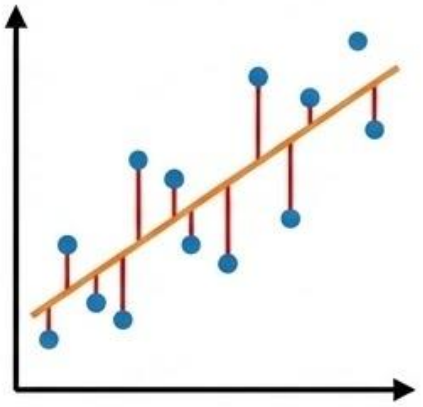
Làm sao cập nhật trọng số? → Dùng **Lan truyền ngược (Backpropagation)**.

HÀM MẤT MÁT (LOSS FUNCTION)

1. Hồi quy

Mean Squared Error (MSE):

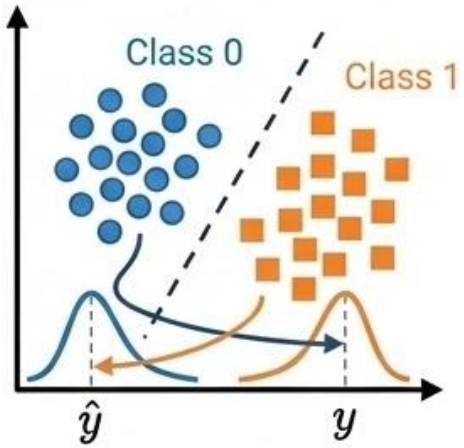
$$J = \frac{1}{m}\sum(\hat{y} - y)^2$$



2. Phân lớp

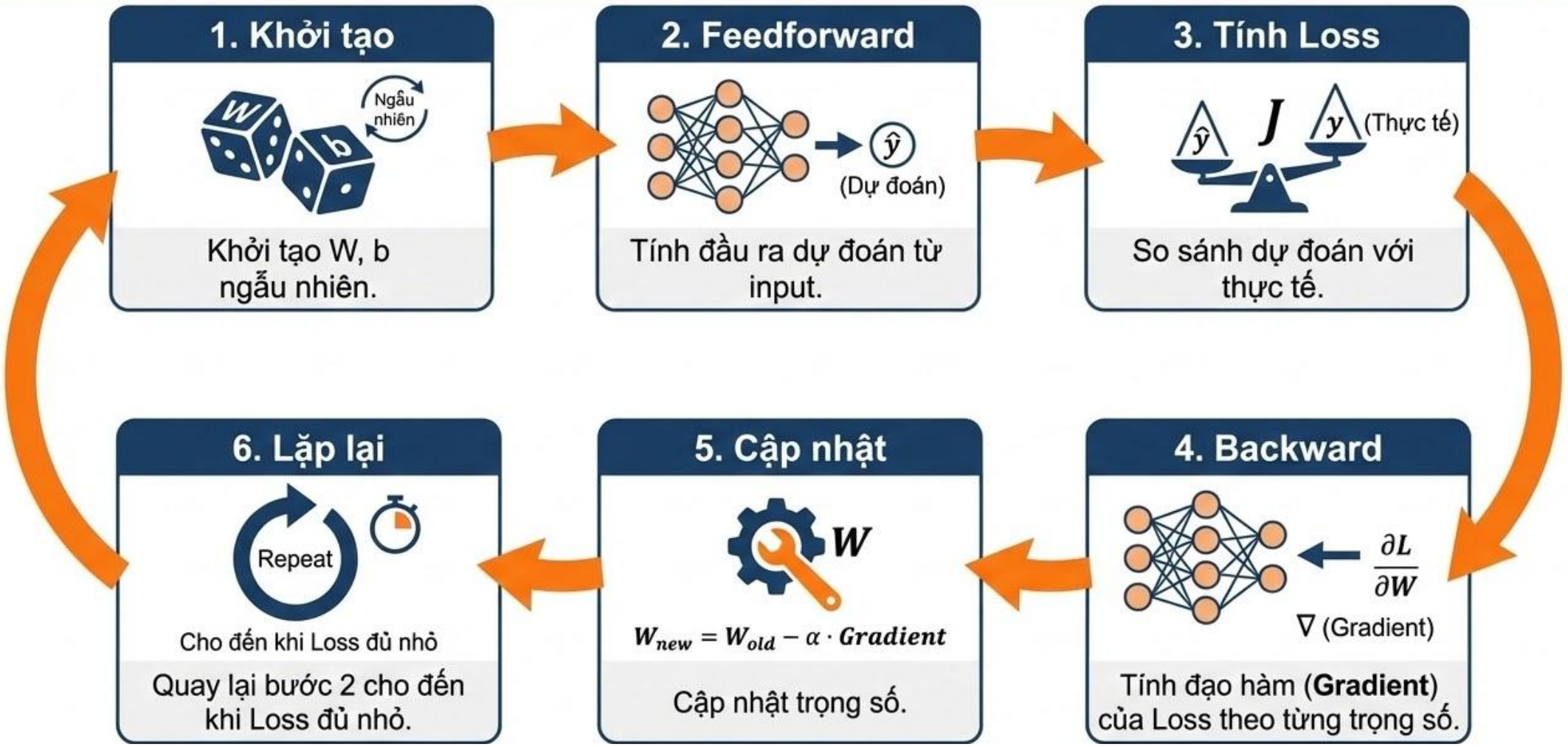
Cross-Entropy Loss:

$$J = -\sum y \log(\hat{y})$$



⬇ **Nguyên tắc:** Hàm Loss càng nhỏ, mô hình càng chính xác.

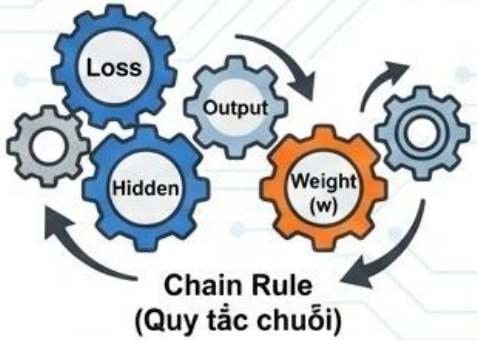
QUY TRÌNH 6 BƯỚC HUẤN LUYỆN

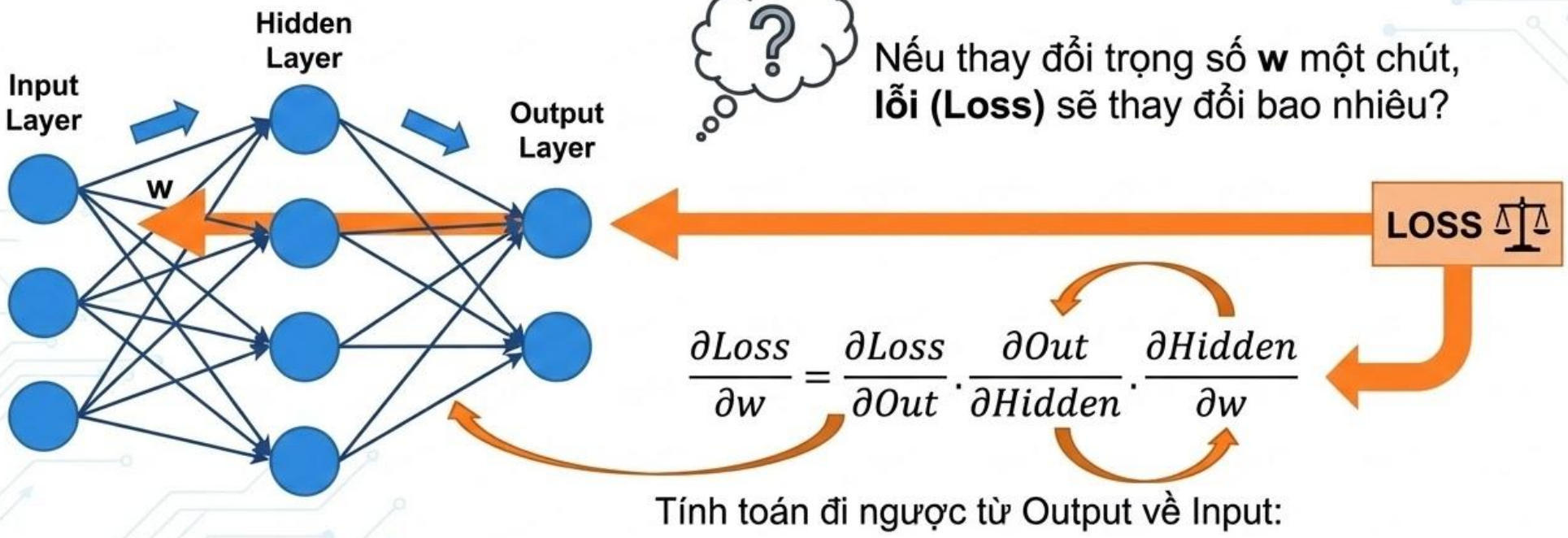


BẢN CHẤT CỦA LAN TRUYỀN NGƯỢC

BẢN CHẤT CỦA LAN TRUYỀN NGƯỢC

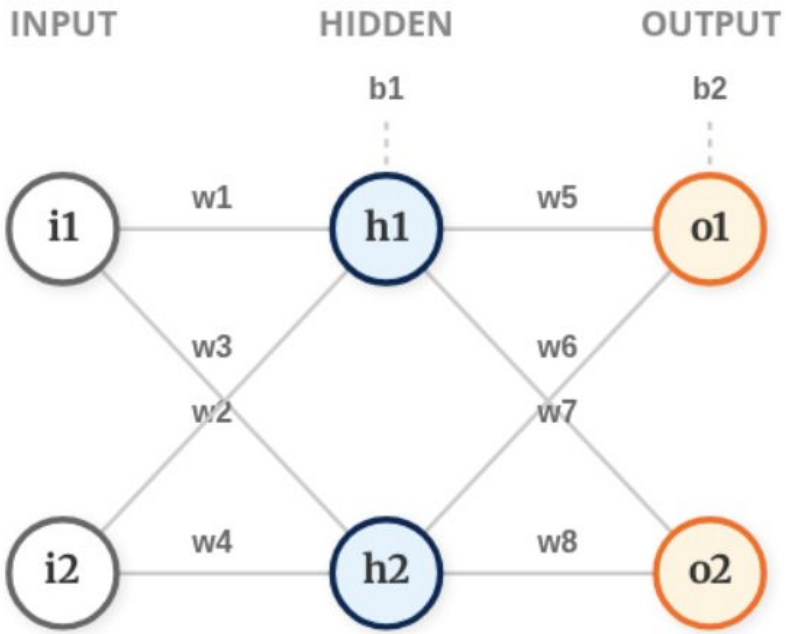
- Là việc áp dụng **Quy tắc chuỗi (Chain Rule)** của đạo hàm.





VÍ DỤ TÍNH TOÁN (SETUP)

- **Input (Đầu vào):**
  i1 = 0.05, i2 = 0.10
- **Target (Mục tiêu):**
  o1 = 0.01, o2 = 0.99
- **Kiến trúc Mạng:**
  2 Input → 2 Hidden → 2 Output
- **Tham số cần học:**
  - 8 Trọng số (w1 -> w8)
  - 2 Bias (b1, b2)
- **Hàm kích hoạt:** Sigmoid



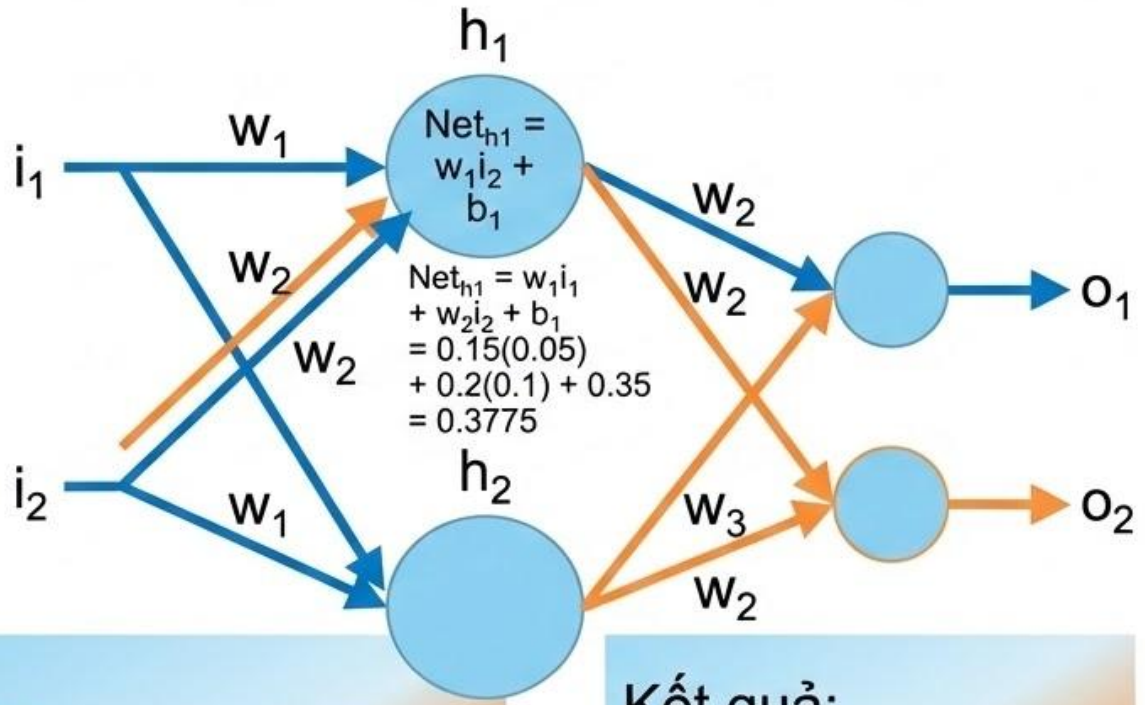
BƯỚC 1: LAN TRUYỀN TIẾN (TÍNH TOÁN)

Tại lớp ẩn $\mathbf{h_1}$:

$Net_{h1} = w_1 i_1 + w_2 i_2 + b_1$
$= 0.15(0.05) + 0.2(0.1) + 0.35$
$= 0.3775$

$\mathbf{Out}_{h1} = \text{Sigmoid}(0.3775) = 0.593$

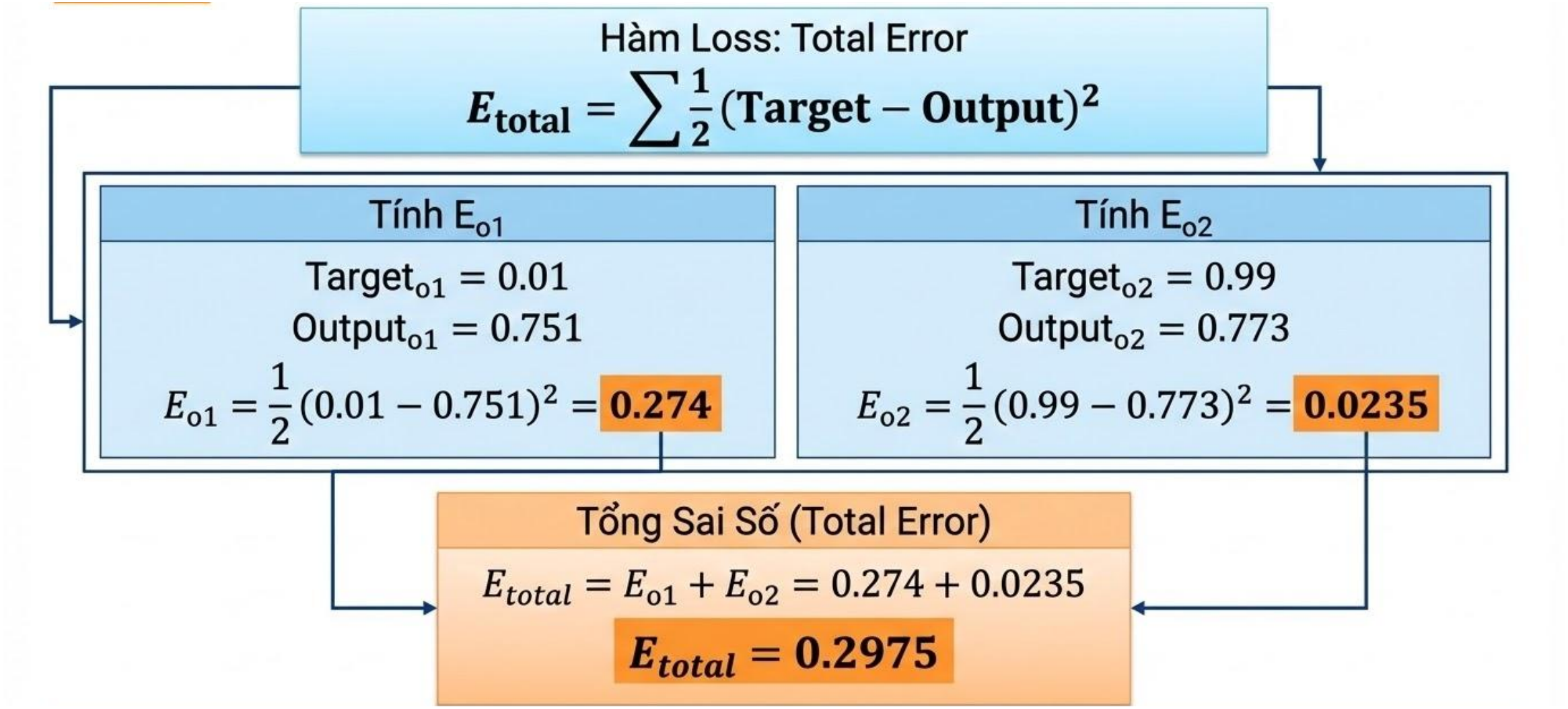
Tương tự tính cho h_2, o_1, o_2.



Tương tự tính cho $\mathbf{h_2}$, o_1, o_2.

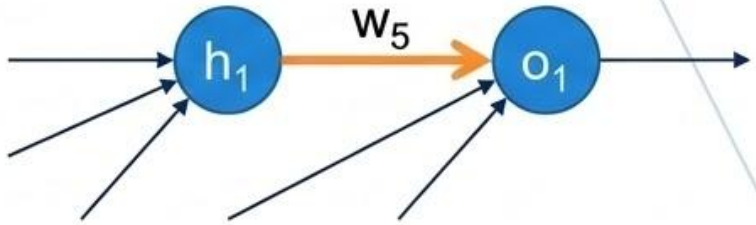
Kết quả:
$\mathbf{Out}_{O1} = 0.751$,
$\mathbf{Out}_{O2} = 0.773$

BƯỚC 2: TÍNH SAI SỐ TỔNG



BƯỚC 3: LAN TRUYỀN NGƯỢC

- Cần cập nhật trọng số w_5 (nối h_1 tới o_1):



- Áp dụng **quy tắc chuỗi (Chain Rule):**

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial Out_{o1}} \cdot \frac{\partial Out_{o1}}{\partial Net_{o1}} \cdot \frac{\partial Net_{o1}}{\partial w_5}$$

1. $\frac{\partial E}{\partial Out}$

$$\frac{\partial E}{\partial Out} = -(Target - Out)$$

(Đạo hàm lỗi theo đầu ra)

2. $\frac{\partial Out}{\partial Net}$

$$\frac{\partial Out}{\partial Net} = Out(1 - Out)$$

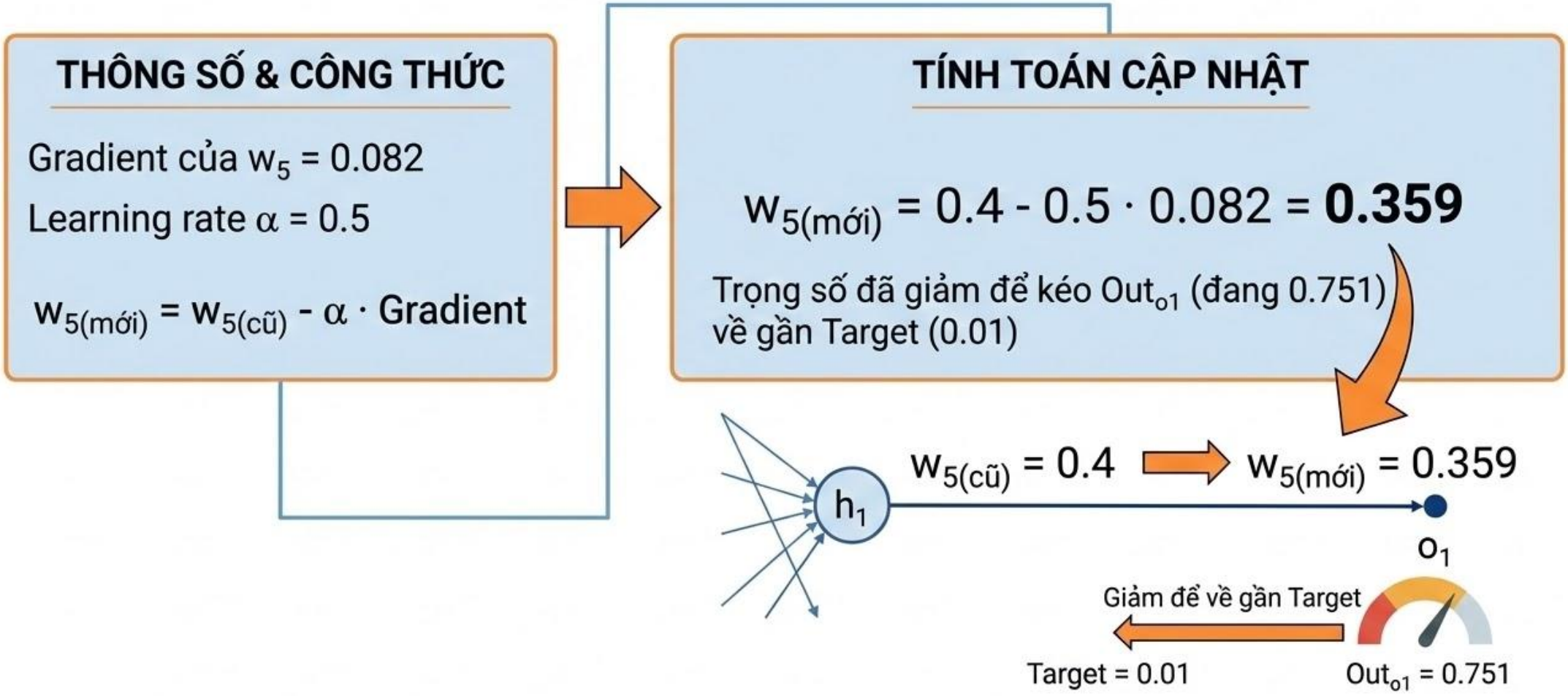
(Đạo hàm Sigmoid)

3. $\frac{\partial Net}{\partial w}$

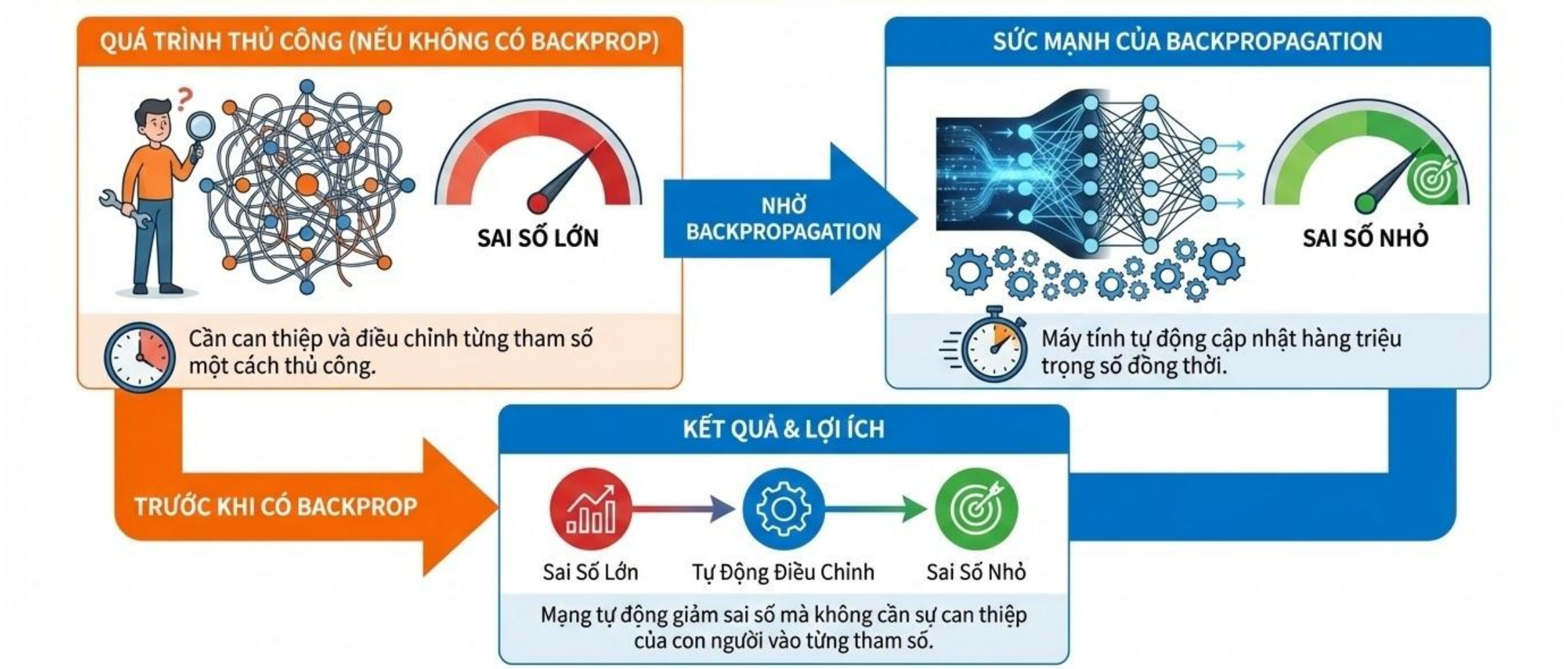
$$\frac{\partial Net}{\partial w} = Out_{h1}$$

(Đạo hàm Net theo trọng số)

BƯỚC 4: CẬP NHẬT TRỌNG SỐ



TỔNG KẾT BACKPROPAGATION



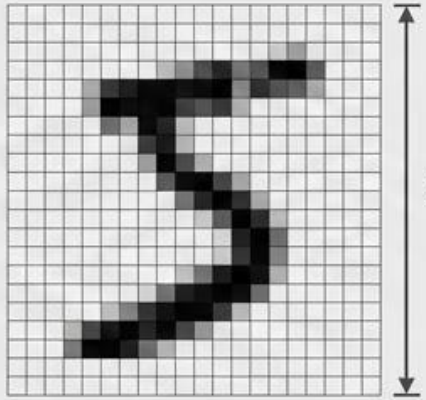
BÀI TOÁN NHẬN DIỆN CHỮ SỐ (MNIST)

Dữ liệu



70.000 ảnh chữ số viết tay (0-9).

Kích thước



Ảnh xám
28 x 28 pixels.

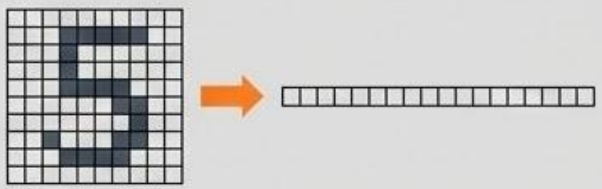
Mục tiêu



Input là ảnh →
Output là nhãn số (0-9).

TIỀN XỬ LÝ DỮ LIỆU

Làm phẳng (Flatten)



Ảnh 2D
(28 x 28)

Vector 1D
(784 phần tử)

Chuyển ảnh 2D (28 × 28) thành vector 1D (784 phần tử).

Chuẩn hóa (Normalize)



Giá trị pixel
[0, 255]

Giá trị chuẩn hóa
[0, 1]

Chia giá trị pixel cho 255 để đưa về khoảng [0, 1]. Giúp mạng hội tụ nhanh hơn.

One-hot Encoding

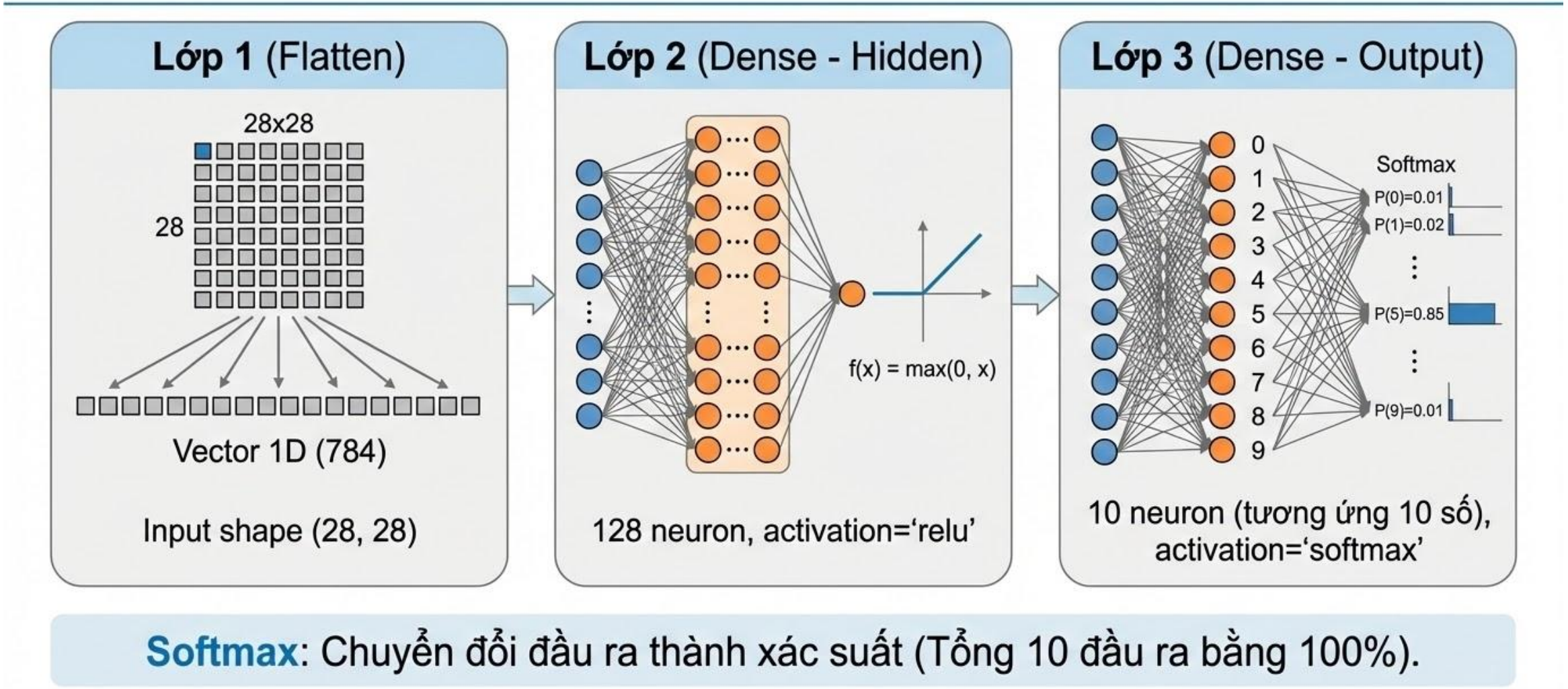
Nhãn
y = 5



Vector One-hot:
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]

Chuyển nhãn y=5 thành vector vector [0,0,0,0,0,1,0,0,0,0].

THIẾT KẾ MẠNG (KERAS)



Softmax: Chuyển đổi đầu ra thành xác suất (Tổng 10 đầu ra bằng 100%).

THIẾT KẾ MẠNG (Code KERAS minh hoạ)

```
# 3) Thiết kế mạng: Flatten -> Dense(128, relu) -> Dense(10, softmax)
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# 4) Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',  # label dạng số nguyên 0..9
    metrics=['accuracy']
)

model.summary()
```

THIẾT KẾ MẠNG (Code KERAS minh hoạ)

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

HUẤN LUYỆN VÀ ĐÁNH GIÁ

Cấu hình

- **Optimizer:** Adam (biến thể tốt hơn của GD).
- **Loss:** Categorical Crossentropy.
- **Metrics:** Accuracy.

Kết quả

- Sau 10 epochs, độ chính xác trên tập kiểm tra có thể đạt > 95%.

HUẤN LUYỆN VÀ ĐÁNH GIÁ

```python
# 5) Train
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8715 - loss: 0.4553 - val_accuracy: 0.9642 - val_loss: 0.1257
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9618 - loss: 0.1309 - val_accuracy: 0.9710 - val_loss: 0.0951
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9748 - loss: 0.0859 - val_accuracy: 0.9743 - val_loss: 0.0878
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9809 - loss: 0.0644 - val_accuracy: 0.9757 - val_loss: 0.0837
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9858 - loss: 0.0473 - val_accuracy: 0.9777 - val_loss: 0.0749
```

HUẤN LUYỆN VÀ ĐÁNH GIÁ

```
# 6) Evaluate trên tập test
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f} | Test accuracy: {test_acc:.4f}")
```

```
Test loss: 0.0802 | Test accuracy: 0.9750
```

HUẤN LUYỆN VÀ ĐÁNH GIÁ

```python
# 7) (Tuỳ chọn) Dự đoán thử vài mẫu và kiểm tra softmax
import numpy as np

probs = model.predict(x_test[:5], verbose=0)  # (5, 10)
preds = probs.argmax(axis=1)

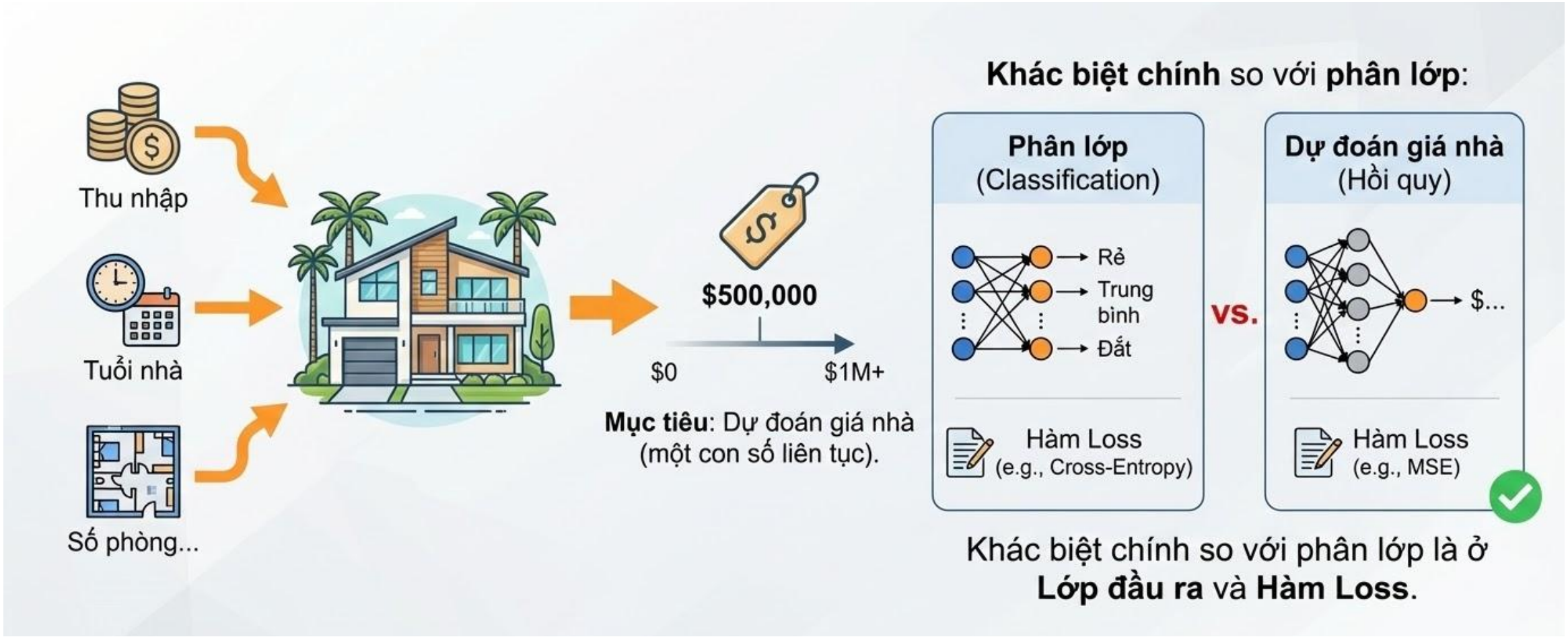
print("Preds:", preds)
print("True :", y_test[:5])
print("Sum probs each sample:", probs.sum(axis=1))  # ~ 1.0
```

```
Preds: [7 2 1 0 4]
True : [7 2 1 0 4]
Sum probs each sample: [1.         0.9999999 0.9999999 1.         1.        ]
```

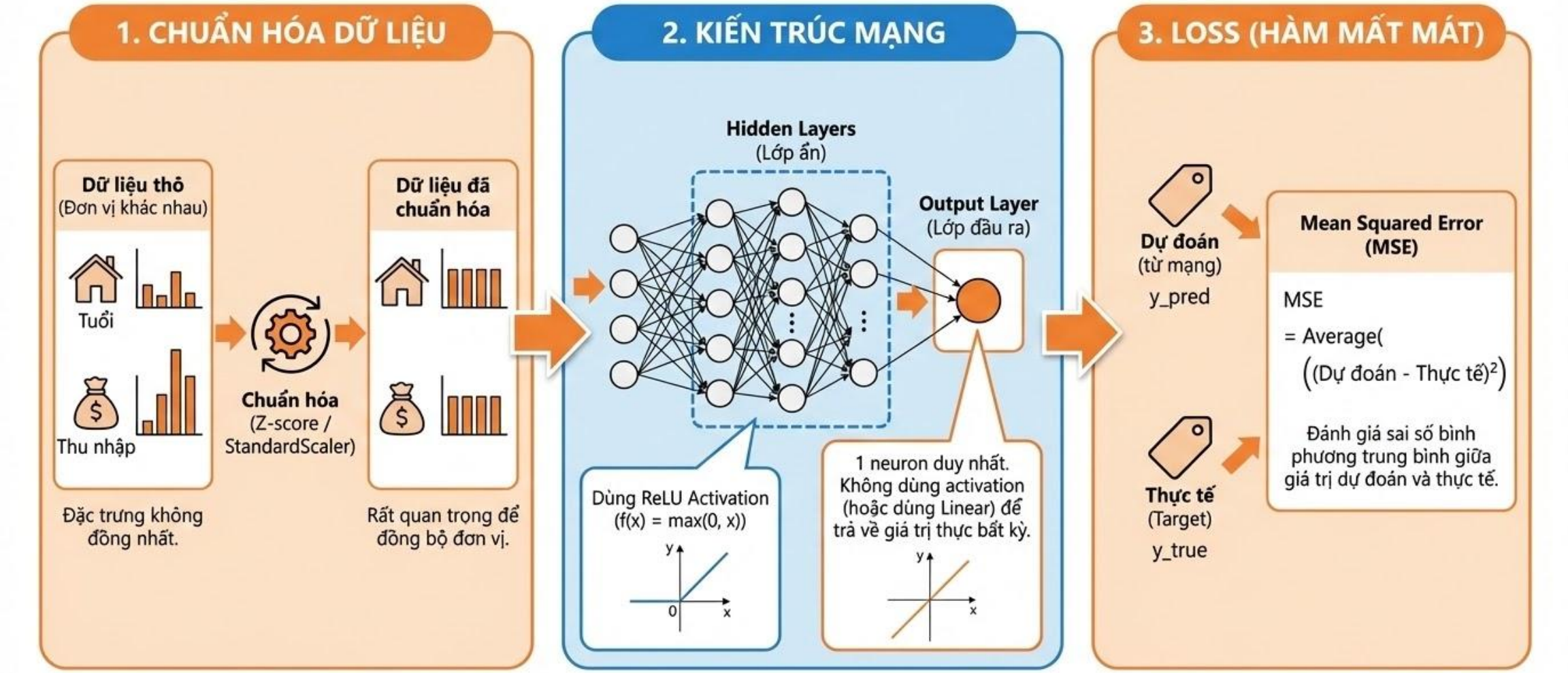
NHẬN XÉT

- Mạng neuron truyền thống (Fully Connected) hoạt động tốt với ảnh nhỏ.
- Với ảnh lớn hơn, ta cần mạng CNN (sẽ học ở Bài 4).

BÀI TOÁN DỰ ĐOÁN GIÁ NHÀ



THIẾT KẾ MẠNG CHO HỒI QUY



ĐÁNH GIÁ MÔ HÌNH HỒI QUY



Accuracy

Không dùng. (Giá lệch 1đ cũng là sai nếu xét Accuracy).



MAE

Mean Absolute Error: Sai số tuyệt đối trung bình (Vd: lệch $500).

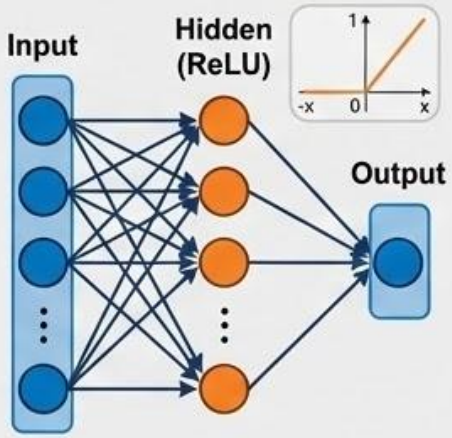


R^2 Score

Độ phù hợp của mô hình (càng gần 1 càng tốt).

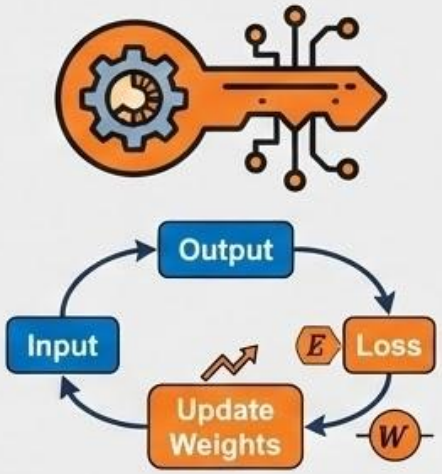
TÓM TẮT BÀI HỌC

1. Cấu trúc



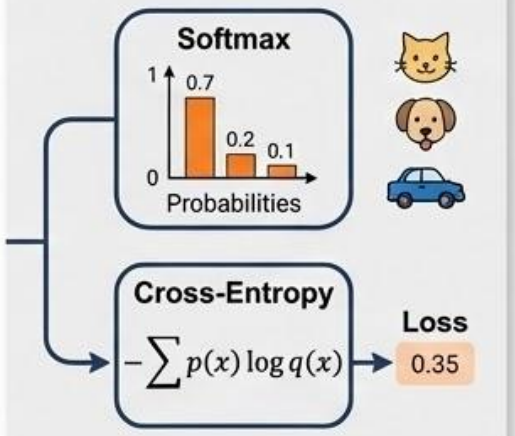
Input → Hidden (ReLU) → Output.

2. Backpropagation



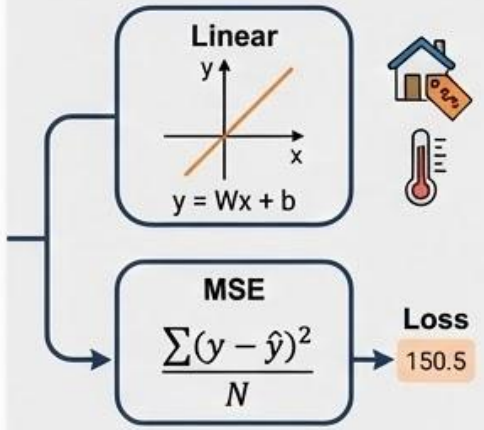
Chìa khóa để huấn luyện mạng sâu.

3. Phân lớp



Output dùng Softmax, Loss dùng Cross-Entropy.

4. Hồi quy



Output dùng Linear, Loss dùng MSE.

CÂU HỎI ÔN TẬP

- Tại sao hàm ReLU tốt hơn Sigmoid cho lớp ẩn?
- Mô tả bằng lời quy trình cập nhật trọng số của mạng neuron.
- Khi nào dùng hàm Softmax?
- **Bài tập:** Cài đặt lại ví dụ phân loại ảnh MNIST bằng Google Colab.